

# Infografía

## Módulo 11 — Capstone: Arena Survival

**Sprites + UI + Shaders, todo junto**

UPB · Gestión I/2026

# Plan de la clase

1. **La idea:** integrar lo aprendido en UN juego, no en demos sueltas.
2. **La columna:** `GameManager` — una máquina de estados que manda.
3. **El fan-out:** un evento (daño) → muchas reacciones, desacopladas.
4. **Sprites:** la FSM de animación que NO se traba (el bug clásico).
5. **Shaders enchufados:** flash, disolver, pausa en gris, viñeta.
6. **Cuatro retos:** paleta · acople · FSM · estado nuevo.

*Es el último módulo antes del 2do parcial: el ensayo general.*

# La idea grande

Hasta ahora cada módulo fue una **demo aislada**: un sprite que anima, un menú, un shader.

Hoy todo eso tiene que **convivir** en un juego jugable — una arena top-down donde sobrevives a oleadas de esqueletos.

El truco: elegir un juego donde los pilares **se necesiten**.

- El HUD y el hit flash reaccionan al **mismo** daño.
- La pausa congela el juego **y** pone la pantalla en gris.
- Todo cuelga de una sola pieza: el **GameManager**.

# La columna: GameManager

Un **autoload** (singleton): existe una sola instancia, accesible desde cualquier escena como `GameManager`. Es la **única fuente de verdad**.

```
enum Estado { MENU, JUGANDO, PAUSA, GAME_OVER }  
  
signal estado_cambiado(nuevo)  
signal vida_cambiada(vida, vida_max)  
signal puntaje_cambiado(puntaje)  
signal oleada_cambiada(oleada)
```

*Regla de oro: todos se SUSCRIBEN a la columna; nadie llama directo a otro sistema.*

# El fan-out: un evento, muchas reacciones

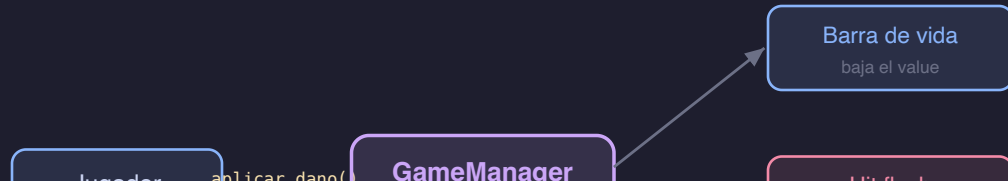
El jugador recibe un golpe. ¿A quién le avisa? **A nadie en particular.** Solo a la columna:

```
func _recibir_golpe(dano):  
    GameManager.aplicar_dano(dano)    # eso es TODO lo que hace
```

`aplicar_dano` emite `vida_cambiada` una vez. Y esa señal la escuchan, por separado:

-  la **barra** del HUD → baja
-  el **flash** del jugador → parpadea
-  la **viñeta** de pantalla → sube si la vida es baja

Un evento, muchas reacciones (fan-out)



# ¿Por qué desacoplar?

**Predice:** si el jugador llamara directo a `hud.barra.value = ...` y a `shader.set_parameter(...)`, ¿qué pasa cuando quieras agregar un cuarto efecto (una sacudida de cámara)?

Con acople directo: tocas el código del **jugador** cada vez.

Con señales: agregas un **suscriptor** nuevo y el jugador ni se entera. El productor del evento no sabe —ni le importa— quién reacciona.

# Sprites: dos máquinas de estado

## En código (la LÓGICA)

IDLE / RUN / ATTACK / HURT / DEATH

decide qué se puede hacer.

## La animación (cómo se VE)

`AnimatedSprite2D` reproduce el clip.

El código dice "ataca", no "frame 14".

La clave: los estados con **loop** (idle/run) se eligen cada frame; los **bloqueados** (attack/hurt/death) se entran una vez y **solo salen** cuando el clip termina.

# El bug clásico: el ataque se traba

Con AnimationTree + "method track" al final del clip, bajo ataques rápidos el clip deja de avanzar y la FSM se WEDGEA — se queda trabada para siempre.

La cura de este módulo: **nada de AnimationTree**. La FSM vive en GDScript y escucha una señal simple del `AnimatedSprite2D`:

```
func _on_animation_finished():  
    if estado in [ATTACK, HURT]:  
        estado = IDLE  
        sprite.play("idle")      # ÚNICA salida de un estado bloqueado
```

Y mientras esté bloqueado: `return` temprano (sin moverse, sin re-`play()`).



# Loop sí, loop no

En `SpriteFrames`, cada clip tiene un check **Loop**:

- `idle`, `run` → **Loop ON**: nunca terminan, nunca emiten `animation_finished`.
- `attack`, `hurt`, `death` → **Loop OFF**: emiten `animation_finished` una vez.

**Predice:** si por error dejas `attack` en Loop ON, ¿`animation_finished` se dispara alguna vez? ¿Qué le pasa al ataque?

*Ese checkbox es, literalmente, medio ejercicio 3.*

# Recortar las hojas (SpriteFrames)

El caballero ( `knight2` ) y el esqueleto vienen como **hojas** (tiras de frames).

- El caballero: grilla pareja, frames de 120×40, fila de abajo.
- El esqueleto: cada animación tiene **su propio ancho** (24, 22, 43, 30, 33 px).

`SpriteFrames` recorta cada hoja por separado — una sola grilla no podría. Es la herramienta hecha para esto (a diferencia de keyframear `frame` en un `AnimationPlayer` ).

# Shaders, ya de juego

Todo del módulo 10, ahora **enchufado** y disparado por las señales de la columna:

| Efecto             | Shader                                         | Lo dispara                              |
|--------------------|------------------------------------------------|-----------------------------------------|
| Hit flash (blanco) | <code>hit_flash</code>                         | <code>vida_cambiada</code>              |
| Disolver al morir  | <code>disolver</code>                          | <code>animation_finished</code> (death) |
| Pausa en gris      | <code>pantalla_fx</code> ( <code>gris</code> ) | <code>estado_cambiado</code> (PAUSA)    |
| Viñeta de daño     | <code>pantalla_fx</code> ( <code>dano</code> ) | <code>vida_cambiada</code> (vida baja)  |

*El puente es siempre el mismo: GDScript escribe un uniform con `set_shader_parameter()` + `tween`.*

# El gotcha del material compartido

Un ShaderMaterial es un Resource COMPARTIDO. Si todos los esqueletos usan el mismo .tres y disueltas a uno... se disuelven TODOS.

Arreglo: `resource_local_to_scene = true` en el material (o `material.clone()` en `_ready`). Así cada esqueleto recibe su propia copia.

**Predice:** el hit flash del jugador usa un material propio. ¿Hace falta marcarlo local?  
(pista: ¿cuántos jugadores hay?)

# La máquina de estados, en escena

MENU → (Jugar) → JUGANDO → (Esc) → PAUSA → (Esc) → JUGANDO  
JUGANDO → (vida 0) → GAME\_OVER → (Reintentar) → JUGANDO

GameManager: la máquina de estados



cada transición emite estado\_cambiado · HUD y shaders reaccionan

*La pausa congela el árbol (get\_tree().paused); el menú sobrevive por su process\_mode.*

# Los tres niveles del proyecto

-  **Demos** — el juego corre de punta a punta: te mueves, atacas, los esqueletos persiguen y se disuelven, el HUD vive, Esc pausa.
-  **En vivo** — tres huecos que completamos en clase: la barra de vida, la pausa en gris, y el spawner de oleadas.
-  **Ejercicios** — cuatro retos, cada uno con estado roto visible y pistas en escalera.

# Reto 1 — Shader de paleta (élite)

Un esqueleto "élite" con su propia paleta, escribiendo el shader desde cero.

```
// Magenta = falta tu código.  
float lum = dot(original.rgb, vec3(0.299, 0.587, 0.114));  
vec3 elite = mix(color_sombra.rgb, color_brillo.rgb, lum);  
COLOR = vec4(elite, original.a);
```

*No inventamos color por pixel: usamos la luminancia para mezclar entre sombra y brillo.*

## Reto 2 — El acople (fan-out)

Un golpe debe mover la barra, flashear el sprite y subir la viñeta — **por señales.**

```
func _ready():  
    vida_cambiada.connect(_actualizar_barra)  
    vida_cambiada.connect(_flash)  
    vida_cambiada.connect(_vineta)
```

**Predice:** si llegan dos golpes en el mismo frame, ¿la barra "salta" una vez o dos?



## Reto 3 — La FSM que no se traba

El ataque se congela. Dos cosas rotas:

1. No se **bloquea** durante el ataque (sigue moviéndose).
2. Nadie lo **cierra** al terminar (`animation_finished` vacío).

```
if estado == ATTACK:          # (1) bloquear
    velocity = Vector2.ZERO; move_and_slide(); return
...
func _on_animation_finished(): # (2) cerrar
    if estado == ATTACK: estado = IDLE; sprite.play("idle")
```

## Reto 4 – Un estado nuevo

Agregar `OLEADA_LIMPIA`: un respiro con cuenta regresiva entre oleadas.

```
enum Estado { JUGANDO, OLEADA_LIMPIA, PAUSA }
```

**Predice:** si pausas DURANTE la cuenta regresiva y luego desparas, ¿a qué estado deberías volver? ¿Tu toggle de pausa lo hace bien?

*La pausa tiene que funcionar desde CUALQUIER estado, no solo desde JUGANDO.*

## El cierre

Sprites + UI + shaders **no son tres temas sueltos**: son piezas de un mismo juego que se hablan por una columna de señales.

Eso —desacoplar por una máquina de estados— es lo que vas a necesitar en el 2do parcial.

A la arena 